2022 IEEE 18th International Conference on Automation Science and Engineering (CASE)
August 20-24, 2022. Mexico City, Mexico

2022 IEEE 18th International Conference on Automation Science and Engineering (CASE) | 978-1-6654-9042-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/CASE49997.2022.9926628

Deep Reinforcement Learning-based Dynamic Bandwidth Allocation in Weighted Fair Queues of Routers

Jinyan Pan[1], Gang Chen[2], Haoran Wu[1], Xi Peng[3], Li Xia[1]

***Abstract*— Motivated by a real problem of service mechanism in the output port of routers, this paper studies the dynamic bandwidth allocation in a $G/G/1/K$ parallel queueing system, where weighted fair queueing (WFQ) scheduling discipline is applied to support differentiated services for different packet queues. The bursty and complicated characteristics of Internet traffic pose a challenge on the analytic solution for dynamic bandwidth allocation, which requires distributional information of traffic patterns. Since the distributional information of Internet traffic is always unavailable and varied with time, we propose a deep reinforcement learning (DRL) framework to train a bandwidth controller by adaptively interacting with the environment. The controller dynamically allocates bandwidth weights among multiple queues according to the instant queue lengths observed. We train the controller with two advanced DRL algorithms, DDPG and SAC, respectively. With real traffic data, experiment results show that our trained controllers achieve a lower average delay and packet loss rate than a rule-based policy. Our proposed WFQ-DRL algorithm is a first attempt to apply RL algorithms in real scenarios of routers, where the system has eight or more queues and a diversity of real traffic without Poisson assumption is applicable.**

I. INTRODUCTION

The rapid development of information society and the resulting ultra-high demand of telecommunication quality have been leading to urgent effort to improve the performance of data transmission queueing systems, e.g., improving throughput [1, 2], reducing delay [3, 4] and packet loss rate [2]. Due to the contradiction between the increasing traffic and limited bandwidth resources, there is an urgent need to make an efficient utilization of bandwidth resources to improve quality of service (QoS). With the advancement of the 5G information society, some communication enterprises have seen the rapidly growing challenges raised from the currently ultra-high communication demand. In 2019, Huawei proposed the so-called "ten mathematical challenges" related to problems in communication networks, the third one of which focuses on how to model the Internet traffic with non-Poisson assumptions, and more importantly, seeks a control policy to improve the performance of queueing systems in routers. Since the environment of Internet is of high-uncertainty, it is challenging to find out an optimal policy for this task, especially in the context of resource limitations, which requires a breakthrough in theoretical and engineering technologies. This paper is motivated by such an optimal control problem for the data transmission system in routers. Cooperated with the Huawei scientists and engineers, we formulate the practical problem as a $G/G/1/K$ parallel queueing system with limited bandwidth.

There are a variety of scheduling disciplines designed for managing differentiated queues, balancing QoS and fairness. WFQ is an important scheduling discipline that is usually applied to provide QoS guarantees and fairness in communication networks. WFQ [5, 6] provides fair bandwidth allocation among differentiated queues by assigning weights, taking variable-length packets into consideration. Considering priority of queues, another discipline, priority queueing, serves the highest priority queue among non-empty queues, resulting in frequent resource starvation for lower priority queues [7]. To avoid bandwidth starvation, deficit round robin (DRR) queueing [8] was proposed, operating in a round-robin way, which is simple and widely implemented in hardware. Since it is not able to implement parallel assignment of bandwidths in engineering, WFQ is also implemented in a round-robin way similar to DRR, but in a more micro level. It is noteworthy that WFQ is modeled as a parallel queueing system, which is beneficial for analysis. Another attractive feature of WFQ is that it is able to provide a guarantee for end-to-end delay when traffic flows are shaped at the edges of a network [9]. In this paper, WFQ scheduling discipline is applied to support differentiated services for different queues.

However, scheduling disciplines, e.g., WFQ, are unaware of the state of queues and cannot schedule bandwidths dynamically. There is a rich body of literature on the WFQ system, the majority of which focuses on performance evaluation [10–12], while little on the optimal control [13]. Moreover, they rely on strong assumptions and simplifications, e.g., Poisson arrival process [10–13]. The queueing model with the Poisson assumption is commonly adopted in existing research since its memoryless property makes it mathematically easy to analyze and obtain performance formulas. However, on one hand, the control policy that based on simplistic arrival models may fail to work in the high-bursty and diverse Internet scenarios. On the other hand, analytic solutions cannot support large-scale networks. They suffer from unaffordable computational complexity. It is mathematically difficult to solve the inverse matrix when the dimensionality of the problem becomes high, e.g., with a large buffer and multiple queues. However, regardless of theoretical difficulties, non-Poisson assumptions can be closer to the characteristics of Internet traffic, and thus have both academic and engineering importance.

[1]School of Business, Sun Yat-Sen University, Guangzhou 510275, P. R. China. xiali5@sysu.edu.cn
[2]School of Management, Guangzhou University, Guangzhou 510006, P. R. China. chengmathcsu@163.com
[3]Theory Lab, Hong Kong Research Center, Huawei Technologies Co. Ltd., Hong Kong, China. pancy.pengxi@huawei.com

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPALLI. Downloaded on September 16,2026 at 19:42:48 UTC from IEEE Xplore. Restrictions apply.

Therefore, we consider a $G/G/1/K$ parallel queueing system with relaxed assumptions. Due to the mathematical intractability of non-Poisson queueing models, we develop a DRL framework to learn the control policy for dynamic bandwidth allocation. Reinforcement learning has been demonstrating its powerful capability to learn and solve dynamic stochastic optimization problems. It is capable to explore the real world and collect the latest information by interacting with the environment without using any distributional information, which is well-suited for the large and complex telecommunication network.

There is a recently emerged literature stream on the application of reinforcement learning (RL) in queueing systems. This research stream can be classified further according to their adopted RL methods, i.e., model-based RL [14] and model-free RL [13, 15–17]. Our paper belongs to the latter research stream that adopts a model-free RL method. Dai and Gluzman extend a Proximal Policy Optimization algorithm to obtain a control policy for server allocation problems in queueing networks [15]. Raeis et al. adopt a Deep Deterministic Policy Gradient algorithm to minimize service rate while giving a probabilistic guarantee for delay [17]. Wang et al. use a Soft Actor-Critic algorithm to minimize packet loss rate in a WFQ system, but without taking delay into consideration [13]. In comparison, we consider multiple objectives in this paper, i.e., reducing both packet loss rate and average delay (equivalently, queue length). Moreover, we adopt multiple algorithms to solve the dynamic bandwidth allocation problem and thus compare their performances.

The contributions of this paper are twofold. Firstly, we formulate the real problem of routers as a $G/G/1/K$ WFQ parallel queueing system, and develop a WFQ-DRL framework for its dynamic bandwidth allocation problem. To the best of our knowledge, our paper is the first work to study the dynamic bandwidth control in parallel queues with WFQ discipline, non-Poisson traffic, and multiple performance metrics. Our trained controller is able to reduce both average delay and packet loss rate while the queue length is the only available information observed. The ingenuity of our WFQ-DRL framework lies in its ability to optimize multi-criteria under the bandwidth constraint. The second contribution is that our work is implemented on the real scenario of Huawei company's routers, where the WFQ has eight or more queues in practice and the non-Poisson traffic is considered, which is not yet studied in either academia or industry. Our WFQ-DRL algorithms are implemented and comprehensively analyzed in real traffic. Our work may shed light on applying data-driven DRL algorithms to adaptively control WFQ parallel queues, which is a desirable feature of modern routers, as proposed by the Huawei company.

II. MODEL DESCRIPTION

We formulate the transmission system in the output port of routers as a $G/G/1/K$ parallel queueing system under the WFQ discipline. The dynamic process of the transmission system is depicted in Fig. 1. Diverse arrival flows arrive at each queue separately and are transmitted later on at a rate

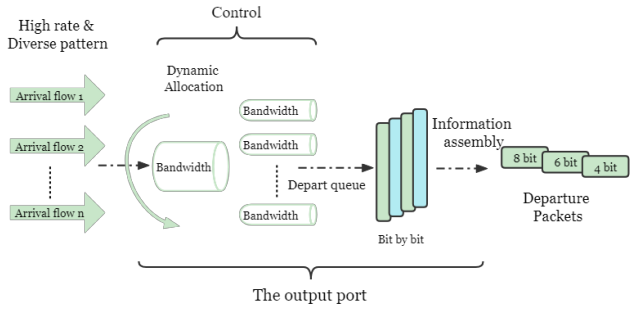


Fig. 1. Illustration of transmission procedure in the output port of a router

depending on the corresponding allocated bandwidth. The total amount of bandwidth resources in the router is fixed, while the arrival rate and pattern of incoming packets are uncertain and vary over time. It is necessary to make an efficient utilization of the limited bandwidth resources by dynamically allocating them among queues with different arrival features, and thus improve the system's performance.

A. Parallel queueing system

As depicted in Fig. 2, our system model is composed of n $G/G/1/K$ queueing systems, sharing common bandwidths in parallel. We assume that packets arrive at the system according to a process with generally distributed interarrival times and then enter a corresponding buffer to wait for service. The arrival rate of flow i is denoted as λ_i. The server i serves packets from the corresponding buffer with generally distributed service times in a FIFO discipline. Its service rate is $\mu_i = w_i \times b$, where w_i is the assigned weight of server i and b is the total bandwidth of the system. The buffer size of queue i is K_i. If the buffer is full, the newly incoming packets will be dropped. Our goal is to find out a control policy that reduces both average delay (equivalently, queue length) and packet loss rate in such a WFQ system.

B. Reinforcement Learning Framework in WFQ

The WFQ system operates under a control policy that allocates bandwidth weights according to the system's dynamic nature. The evolution process of the WFQ system can be formulated as a Markov decision process (MDP) $< S, A, P, R >$, where S, A, P and R denote the state space, action space, state transition probability matrix, and reward

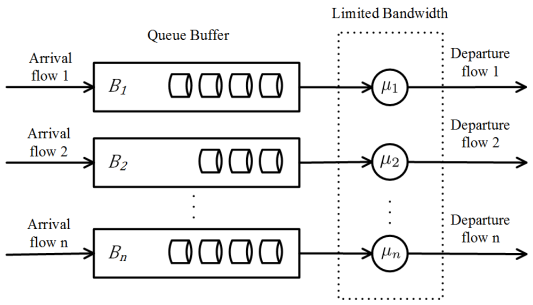


Fig. 2. System model: the $G/G/1/K$ parallel queueing system

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPALLI. Downloaded on September 16,2026 at 19:42:48 UTC from IEEE Xplore. Restrictions apply.

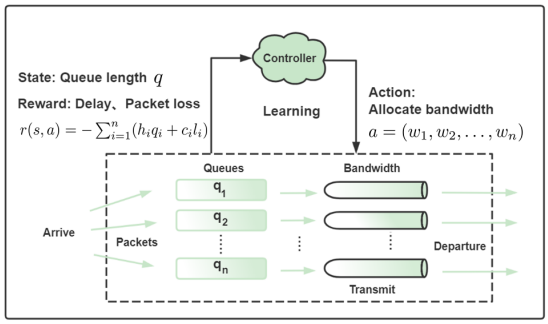


Fig. 3. Interaction between the controller and the queues in a router

function, respectively. To solve the above MDP problem, we introduce an RL framework to train a controller that explores the optimal policy by interacting with the system. Under the non-Poisson assumption, the queue length is just a part of a Markovian state's information. However, we focus on the performance of policy given by RL while only the queue length information is available.

Fig. 3 illustrates the interaction between the controller and the system. At a decision time, the controller observes the system's state, i.e., queue lengths, and then takes an action to assign bandwidth weights. The queueing system operates under the assigned bandwidths until the next decision time, and then feeds back a reward for the controller to update its policy. With respect to the decision time, we consider two different types of dynamic control problem in this paper, i.e., a state-dependent problem and a slot-dependent problem. The decision time of the former one occurs when the system's state changes, i.e., a packet arrives or a service is completed. In this context, the control policy is a strictly state-based dynamic policy. It outputs a new action once the observed state changes.

However, it is costly and infeasible to frequently adjust the bandwidth weights in a large telecommunication network. Thus, we also consider a slot-dependent problem, in which the decision time occurs every t seconds. It is more feasible to implement in engineering, but also more difficult to obtain an optimal policy, since the state may change a lot within a given time slot. Both problems can be modeled as an RL problem with a countable state space and a continuous action space as follows.

- **State:** Let q_i denotes the number of packets waiting in the queue i or being served in the server i. The system's state is $q = (q_1, q_2, \ldots, q_n)$, $i = 1, 2, \ldots, n$. It is easy to capture queue lengths, which provides feasibility for the implementation of our RL method in telecommunication networks. We assume that each queue has a finite buffer K_i, s.t., $q_i \leq K_i$. The state space is denoted as $\mathcal{S} = \{q = (q_1, q_2, \ldots, q_n) \in \mathbb{N}^n : q_i \leq K_i\}$.

- **Action:** The action a is defined as the bandwidth weights assigned to each queue at the decision time. The action space is denoted as $\mathcal{A}$, where $\mathcal{A} = \{a = (w_1, w_2, \ldots, w_n) \in \mathbb{R}_+^n : \sum_{i=1}^{n} w_i = 1\}$. We define a stationary Markovian policy π as a mapping from the state space to the action space: $\pi : \mathcal{S} \to \mathcal{A}$.

- **Reward:** As can be derived from Little's law [18], minimizing average delay is equivalent to minimizing average queue length in queueing systems. We assume that each packet in queue i incurs a holding cost h_i per unit time, which is equivalent to imposing a penalty on its delay. The number of packets dropped in a unit time is denoted as l_i. Each dropped packet also brings a penalty c_i. We derive the reward function, which includes penalties of queue length q_i and packet loss l_i, $i = 1, 2, 3, ..., n$.

$$r(q, a) = -\sum_{i=1}^{n}(h_i q_i + c_i l_i).$$

- **Objective:** Our goal is to obtain an optimal stationary Markovian policy π that minimizes both queue length q_i and packet loss l_i in the long-run average cost:

$$\min_{\pi} \lim_{T \to \infty} \frac{1}{T} \mathbb{E}_{\pi} \int_0^T \sum_{i=1}^{n}(h_i q_i + c_i l_i) dt.$$

III. WFQ-DRL ALGORITHM

To dynamically allocate bandwidths, we develop a deep reinforcement learning framework for weighted fair queues (WFQ-DRL). Under the WFQ-DRL framework, a controller recursively interacts with the queueing system in an episodic manner to explore its dynamics. By explorations, the controller updates its knowledge about the environment and then exploits it to obtain a higher reward. One of the difficulties is that the multiple queues and large buffer characteristics in the WFQ system lead to the curse of dimensionality. To tackle this problem, we adopt DRL algorithms and fit the estimation of the value function and control policy into neural networks' parameters, which is able to avoid the requirement of large storage space. We convert the system's state q to a binary vector s as the input of neural networks, and thus obtain a normalized form of state information for different buffers, which can avoid a gradient exploding problem while computing policy gradient. Thus, the dimension of s is $\sum_{i=1}^{n} \lceil \log_2(K_i + 1) \rceil$.

Moreover, in order to obtain a high utilization efficiency of data, we adopt off-policy DRL algorithms. Technically, a replay buffer is used in the off-policy algorithm to break the correlation between sequential data. Therefore, it has the capability to make repeated use of historical data and thus improve the data utilization efficiency. After each step of exploration, the collected interaction data $(s_t, a_t, r_{t+1}, s_{t+1})$ are stored in the replay buffer to be sampled later on to update the control policy. The unified framework for the dynamic bandwidth allocation problem in WFQ system is given in Algorithm 1.

Specifically, we adopt two advanced DRL algorithms, namely, DDPG and SAC, see Algorithm 2 and 3. Since our action space $\mathcal{A}$ is continuous, we first adopt a Deep

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPALLI. Downloaded on September 16,2026 at 19:42:48 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 The DRL framework for WFQ

Input: State space $\mathcal{S}$, action space $\mathcal{A}$, max episode E, episode's time length T, time slot Δt and DRL algorithm $algo$.
Initialization: Initialize parameters of neural networks in $algo$.
For $episode\ e = 0, 1, 2, \ldots, E$ **do**
  Initialization: Environment: $q_0 = (q_0^1, q_0^2, ..., q_0^n)$. Action exploration process $\mathcal{N}$ according to $algo$.
  Normalize: Convert q_0 to a binary vector s_0.
  While $t < T$:
    Take Action: Derive $a_t = (w_t^1, w_t^2, ..., w_t^n)$ such that $\sum_{i=1}^{n} w_t^i = 1$, using policy network of $algo$ and exploration process $\mathcal{N}$.
    Operate: Operate the queueing system under a_t.
    Reward: $r_{t+1} = \sum_{i=1}^{n} h_i q_{t+1}^i + \sum_{i=1}^{n} c_i l_{t+1}^i$.
    Observe: Observe queue length: $q_{t+1} = (q_{t+1}^1, q_{t+1}^2, q_{t+1}^3, ..., q_{t+1}^n)$.
    Normalize: Convert q_{t+1} to a binary vector s_{t+1}.
    Store: Store transition $(s_t, a_t, r_{t+1}, s_{t+1})$ in $\mathcal{D}$.
    Update: Sample $(s_k, a_k, r_{k+1}, s_{k+1})$ from $\mathcal{D}$. Update parameters of neural networks using the sampled gradient.
    $t \leftarrow t + \Delta t$.

Algorithm 2 The DDPG Algorithm for WFQ

Initialization: Initialize the parameters of actor network μ, critic network Q, target actor network $\bar{\mu}$ and target critic network $\bar{Q}$, i.e., $\phi, \theta, \bar{\phi}, \bar{\theta}$, respectively.
For each episode **do**
  Initialization: Initialize process $\mathcal{N}$ to produce a Ornstein-Uhlenbeck Noise for each action exploration.
  For each environment step **do**
    Store Data:
    Take Action: $a_t = \mu(s_t|\phi) + \mathcal{N}_t$.
    Collect transition: $(s_t, a_t, r_{t+1}, s_{t+1})$.
    Store $(s_t, a_t, r_{t+1}, s_{t+1})$ in replay buffer $\mathcal{D}$.
    Update Parameters:
    Randomly sample a minibatch of m transitions $(s_k, a_k, r_{k+1}, s_{k+1})$ from $\mathcal{D}$.
    Update critic and actor network by minimizing the loss, respectively:

$$J_Q(\theta) = \frac{1}{m} \sum_{k=1}^{m} \frac{1}{2} [r_{k+1} + Q_{\bar{\theta}}(s_{k+1}, \mu_{\bar{\phi}}(s_{k+1})) - Q_\theta(s_k, a_k)]^2.$$

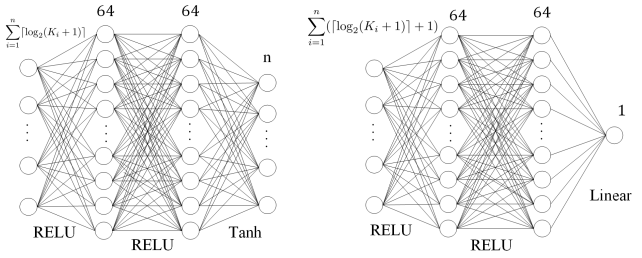
$$J_\mu(\phi) = \frac{1}{m} \sum_{k=1}^{m} Q_\theta(s_k, \mu_\phi(s_k)).$$

    Update the parameters of Q and μ:
    $\theta \leftarrow \theta - \lambda_Q \nabla_\theta J_Q(\theta)$.
    $\phi \leftarrow \mu - \lambda_\mu \nabla_\phi J_\mu(\phi)$.
    Update the parameters of $\bar{Q}$ and $\bar{\mu}$:
    $\bar{\theta} = \tau\theta + (1-\tau)\bar{\theta}$.
    $\bar{\phi} = \tau\phi + (1-\tau)\bar{\phi}$.

Deterministic Policy Gradient (DDPG) [19] algorithm to learn the optimal policy. DDPG is a classic deterministic policy algorithm designed for continuous control problems. In a problem with continuous action space, it is more sample efficient for a deterministic policy algorithm to learn than a stochastic one. It mainly results from the computation of policy gradient. Specifically, the deterministic policy outputs a deterministic action for a given state, while a stochastic one outputs the probability of the action set. Therefore, when computing the policy gradient, a stochastic policy algorithm has to sample from the action space and then computes the integration as Algorithm 3.5 shows. However, as shown in Algorithm 2.6, the deterministic one does not have to do so, and thus improves the sample efficiency. The implementation details of DDPG are presented in Algorithm 2.

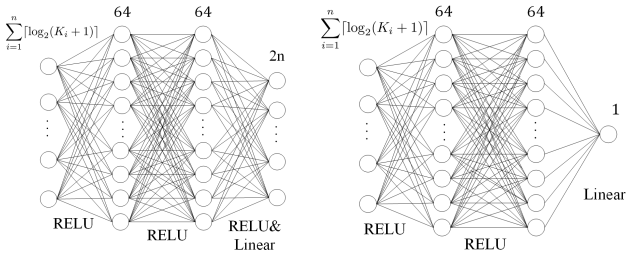
DDPG uses a actor network and a critic network to approximate its deterministic policy and value function, respectively. Moreover, it uses a target actor network and a target critic network to stabilize the learning process. Their parameters are the same with actor and critic network, respectively, updated softly in a moving average way. Fig. 4 depicts the layers, neurons, and activation functions of the actor network and critic network.

We also adopt another advanced algorithm named Soft Actor-Critic (SAC). It maximizes the entropy of policy while maximizing the expected reward [20]. By maximizing the entropy, SAC improves its exploration efficiency. It has been proven to achieve stable performance in a continuous action space. SAC uses policy network π, action value network Q and value network V to approximate its stochastic policy, action value function and value function, respectively. Fig. 5 depicts the structures of π network and V network. The



(a) actor network. (b) critic network.

Fig. 4. Structure of neural networks in DDPG.



(a) policy network. (b) value network.

Fig. 5. Structure of neural networks in SAC.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPALLI. Downloaded on September 16,2026 at 19:42:48 UTC from IEEE Xplore. Restrictions apply.

Q network's structure is the same as the critic network in DDPG. The π network outputs the mean and variance of the action space's distribution instead of a deterministic action, which is different from DDPG. SAC uses a target value network $\bar{V}$ to achieve a stable learning process. It has the same structure as V with softly updated parameters. It is worthwhile to compare the performance between DDPG and SAC. The details of SAC are presented in Algorithm 3.

Algorithm 3 The SAC Algorithm for WFQ

Initialization: Initialize the parameters of networks π, Q, V and $\bar{V}$, i.e., $\phi, \theta, \psi, \bar{\psi}$, respectively.
For each episode **do**
  For each environment step **do**
    Store Data:
    Take Action: $a_t \sim \pi_\phi(a_t|s_t)$.
    Collect transition: $(s_t, a_t, r_{t+1}, s_{t+1})$.
    Store $(s_t, a_t, r_{t+1}, s_{t+1})$ in replay buffer $\mathcal{D}$.
    Update Parameters:
    Randomly sample a minibatch of m transitions $(s_k, a_k, r_{k+1}, s_{k+1})$ from $\mathcal{D}$.
    Minimizing loss of V, Q and π, respectively:

$$J_V = \frac{1}{m}\sum_{k=1}^{m} \frac{1}{2}\{\mathbb{E}_{a_k\sim\pi_\phi}[Q_\theta(s_k,a_k) - \log \pi_\phi(a_k|s_k)] - V_\psi(s_k)\}^2.$$

$$J_Q = \frac{1}{m}\sum_{k=1}^{m} \frac{1}{2}[r_{k+1} + \gamma V_{\bar{\psi}}(s_{k+1}) - Q_\theta(s_k,a_k)]^2.$$

$$J_\pi = \frac{1}{m}\sum_{k=1}^{m} [\log \pi_\phi(f_\phi(\epsilon_k;s_k)|s_k) - Q_\theta(s_k, f_\phi(\epsilon_k;s_k))].$$

    Update the parameters of V, Q and π:
    $\psi \leftarrow \psi - \lambda_V \nabla_\psi J_V(\psi)$.
    $\theta \leftarrow \theta - \lambda_Q \nabla_\theta J_Q(\theta)$.
    $\phi \leftarrow \phi - \lambda_\pi \nabla_\phi J_\pi(\phi)$.
    Update the parameters of $\bar{V}$:
    $\bar{\psi} = \tau\psi + (1-\tau)\bar{\psi}$.

IV. EXPERIMENT

In this section, we present evaluations of different algorithms for the bandwidth optimization task in the WFQ system. We also compare the performances of our trained controllers with rule-based methods. The experiments are implemented in python 3.8 using pytorch. The implementation parameters are as Table I.

TABLE I
EXPERIMENT IMPLEMENTATION PARAMETERS

Parameters	Value
CPU	Two AMD Ryzen threadripper pro 3975wx 32-cores
Memory	128 GB
Learning rates	$\lambda_\mu, \lambda_\pi = 10^{-4}; \lambda_V, \lambda_Q = 10^{-3}$
Updating rate	$\tau = 10^{-2}$
Discount factor	$\gamma = 0.99$
Replay buffer	10000
Batch size	$m = 128$
OU Noise	$\mu = 0, \theta = 0.15, \sigma = 0.05$

A. State-dependent Problem with Two Queues

We first consider a two-parallel queueing system where packets arrive depending on two corresponding arrival processes with arrival rates $(\lambda_1, \lambda_2) = (0.2, 0.4)$ $packet/s$. Each queue has a finite buffer, $K_1 = K_2 = 32$ $packet$. In routers, the service time of a packet is determined by its packet size and assigned bandwidths. We assume that the average packet sizes of the two queues are $plen_1 = plen_2 = 1$ $byte/packet$. The total bandwidth is $b = 1$ $byte/s$. The control policy allocates bandwidth weights w_1, w_2 to two servers while satisfying $w_1 + w_2 = 1$, every time the system's state changes. After that, the system operates under the service rate vector $(b \times w_1, b \times w_2)$ $byte/s$ until the next decision time. We generate inter-arrival times and service times by exponential distributions and run 120 episodes with a time length of $2000s$ per episode to train controllers.

We compare the performances of the DDPG with a rule-based policy, called $c\mu$-rule. The $c\mu$-rule chooses to serve the queue which has the largest expected cost decrease per slot until it is empty. It updates action once the system's state changes, which is suitable for our state-dependent problem. In a state-dependent problem with $MAP/M/1$ queue, the optimal control of service rate is a bang-bang control [21]. Moreover, the $c\mu$-rule [22] has been proven to be an optimal policy for assigning service attention between competing queues with infinite buffer, i.e., without considering the packet loss metric. Consistently, there is no packet loss in this experiment since the buffer is set to be large enough. That is to say, the $c\mu$-rule is an optimal policy for the setting of this experiment. We set the holding cost as $h_1 = 0.3, h_2 = 0.7$. Given that the average packet sizes $plen_1 = plen_2 = 1$, which means the service rates of two servers are the same when they are assigned with a same bandwidth weight. Therefore, the $c\mu$-rule can be implemented as allocating all the bandwidth to queue 2 until it becomes empty, since the holding cost $h_1 < h_2$. The reward function is:

$$r(q,a) = -(0.3 \times q_1 + 0.7 \times q_2).$$

Fig. 6 plots the results of our simulation experiment. We train controllers with four different random seeds. The average performances and standard error bands are plotted by

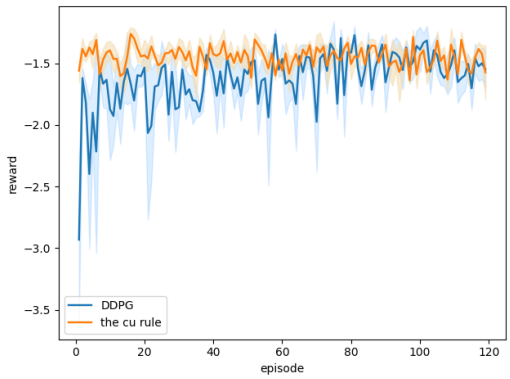


Fig. 6. Comparison between DDPG and the $c\mu$-rule

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPALLI. Downloaded on September 16,2026 at 19:42:48 UTC from IEEE Xplore. Restrictions apply.

the dark curves and pale regions in Fig. 6, respectively. Note that the $c\mu$-rule is an optimal policy for this simple scenario. Fig. 6 shows that the long-run average reward of controllers trained by DDPG quickly converges to the $c\mu$-rule.

B. Slot-dependent Problem with Two Queues

We now turn to a slot-dependent problem which is more feasible and less costly in engineering implementation for large telecommunication networks. The control policy here allocates bandwidth weights w_1, w_2 to two servers every $\Delta t = 20$ seconds.

We consider a heavy load system with two queues. The arrival rates are increased to $\lambda_1 = 0.3\ packet/s$, $\lambda_2 = 0.6\ packet/s$. The inter-arrival times and service times are generated by gamma distribution. Their Squared Coefficient of Variation (SCV) are both set as 0.8. We assume that the average sizes of packets from queue 1 and queue 2 are $plen_1 = plen_2 = 1\ byte/packet$. The total bandwidth is $b = 1\ byte/s$. The queueing system in this experiment is heavy loaded, since the load condition $\frac{(\lambda_1+\lambda_2)}{b} = 0.9$ is close to 1. Meanwhile, each queue has a finite buffer, $K_1 = K_2 = 32\ packet$. The heavy load and finite buffer of the system result in a phenomenon of packet loss.

We set the holding cost as $h_1 = 0.5, h_2 = 0.5$ and the penalty of packet loss as $c_1 = 1, c_2 = 1$. The reward function is:

$$r(q, a) = -(0.5 \times q_1 + 0.5 \times q_2 + l_1 + l_2).$$

In this experiment, we run 200 episodes with a time length of $2000s$ per episode to train bandwidth controllers with five different random seeds. We compare the system performances of DDPG with SAC in the training process. Fig. 7 plots the convergence of long-run average reward and average packet loss rate, respectively. Fig. 8 plots the convergence of average queue length and average delay, respectively.

It can be observed from Fig. 7 and Fig. 8 that the DDPG and SAC algorithms both quickly converge, reducing the packet loss rate, average queue length and average delay. Besides, SAC converges more quickly than DDPG, which is consistent with the previous discussion that SAC has a higher exploration efficiency. The performance of SAC is more stable than DDPG, with a smaller variation. Moreover, Fig. 8 shows that the average delay can be greatly reduced while the reward function imposes a penalty on the queue lengths, which is consistent with the Little's Law. That is, reducing the queue length in the buffer would coherently decrease the queueing delay.

C. Slot-dependent Problem with Eight Queues

To be closer to the large telecommunication network, we now consider a more complex WFQ system with eight queues. Typically, each output port in routers consists of eight different queues. We conduct an experiment on eight simulation arrival flows with arrival rates $\lambda = (0.03, 0.16, 0.2, 0.1, 0.08, 0.15, 0.06, 0.05)\ packet/s$ and generate its inter-arrival times and service times by

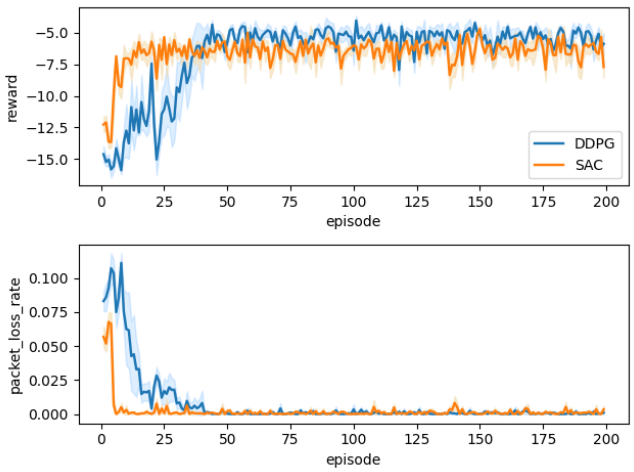


Fig. 7. Convergence of long-run average reward and packet loss rate

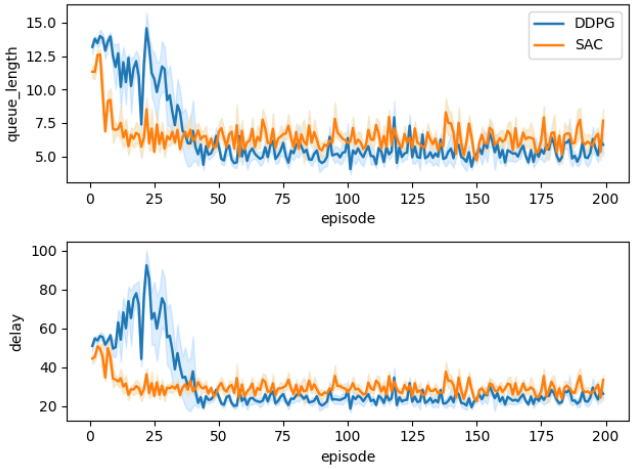


Fig. 8. Convergence of average queue length and delay

gamma distributions with a SCV of 0.8. The average packet sizes are $plen_i = 1\ byte/packet$, $i = 1, 2, \cdots, 8$. Their buffers are $K_1 = K_2 = \cdots = K_8 = 32\ packet$. The total bandwidth is $b = 1\ byte/s$. The controller takes an action every $\Delta t = 20$ seconds. We set the holding cost as $h_i = \frac{1}{8}$ and the penalty of packet loss as $c_i = 1, i = 1, 2, \ldots, 8$. The reward function is :

$$r(q, a) = -\sum_{i=1}^{8}(\frac{1}{8} \times q_i + l_i).$$

In this experiment, we run 1000 episodes with a time length of $2000s$ per episode to train bandwidth controllers with five different random seeds. Fig. 9 plots the long-run average reward of the eight queues system in the training process of DDPG and SAC. Both algorithms converge after 200 episodes. The variation of SAC is smaller than DDPG, which is similar to the results in section 4.2. Furthermore, from the comparison, we find that DDPG can achieve a better performance than SAC after convergence, which is consistent with our discussion in Section 3 that DDPG has a higher

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPALLI. Downloaded on September 16,2026 at 19:42:48 UTC from IEEE Xplore. Restrictions apply.

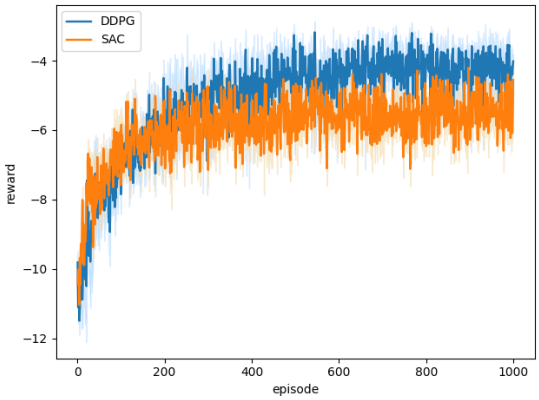


Fig. 9. Comparison between DDPG and SAC in eight queues

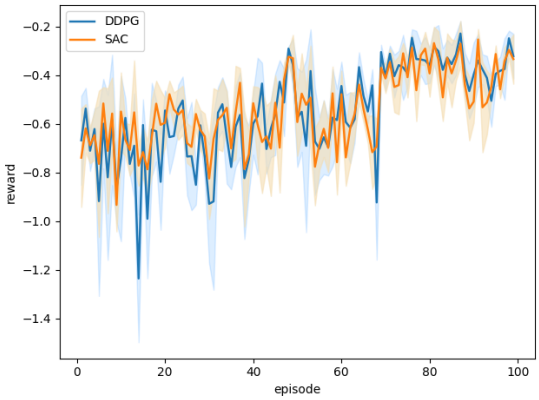


Fig. 10. Training process of DDPG and SAC in real traffic

sample efficiency.

D. Application to Real Traffics

To demonstrate the effectiveness of our proposed WFQ-DRL framework for real implementations, we conduct an experiment using traffic data from a real telecommunication network. The traffic patterns in real routers are more complex than the distributions used in simulation experiments. We assume that the decision time in this experiment occurs every Δt seconds.

Training In this experiment, eight arrival flows with diverse traffic patterns arrive at the WFQ system. We use five groups of traffic data to train the controllers. The average arrival rates and packet sizes of eight flows in these five groups are shown in Table II. The reward function is :

$$r(q,a) = -\sum_{i=1}^{8}(\frac{1}{8} \times q_i + l_i).$$

The episode's time length, time slot, buffers and total bandwidth resources are $T = 20000\ \mu s$, $\Delta t = 200\ \mu s$, $K_1 = K_2 = \cdots = K_8 = 16\ packet$, $b = 8\ byte/\mu s$, respectively. We run 100 episodes to train the controllers by DDPG and SAC algorithms. The training process is depicted in Fig. 10.

Test Now, we use another five groups of test data from the real telecommunication network to compare the performance of our trained controllers with a rule-based policy. Since the unit costs of each queue in this experiment are the same, we adopt another rule-based method called longest connected queue (LCQ)[23] as a baseline of comparison, instead of the $c\mu$-rule. The LCQ rule is to serve the longest connected queue in a single server system with n parallel queues. In our WFQ problem, the LCQ rule can be implemented as allocating all the bandwidth resources to the longest queue. It has been proven to minimize the delay when the arrival and service statistics of different queues are identical and the buffer is infinite [23]. However, our WFQ system has a finite buffer and different arrival statistics. Most importantly, the decision time in this experiment occurs every Δt seconds instead of every time the state changes, which is different from the original LCQ rule's setting. Therefore, the LCQ rule is not an optimal policy for our WFQ system.

TABLE II

AVERAGE STATISTICS OF FIVE GROUPS

Queues	**Average Arrival Rates** $(packet/\mu s)$	**Average Packet lengths** $(byte/packet)$
1	3.40E-03	132.73
2	2.24E-03	130.30
3	3.78E-03	119.95
4	2.45E-03	121.08
5	6.30E-04	128.55
6	2.96E-04	133.77
7	2.59E-04	125.23
8	2.10E-04	121.28

Fig. 11 shows the performances of the LCQ rule and the controllers trained by DDPG and SAC, respectively. The horizontal dashed line and shaded area depict the average performance and variation of each policy, respectively. It can be observed from Fig. 11 that the SAC and DDPG algorithms both outperform the LCQ rule. Besides, the performance of controllers trained by DDPG is slightly better than that trained by SAC, which is similar to the results in section 4.3.

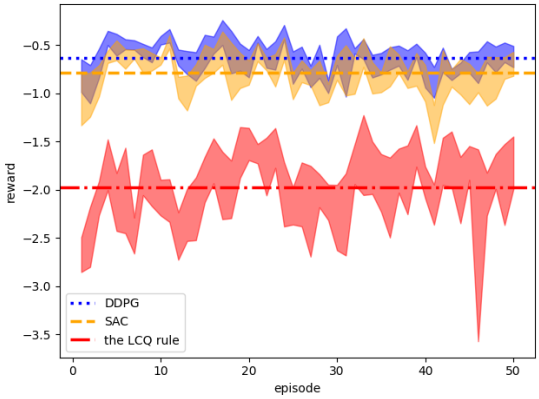


Fig. 11. Comparison of DDPG, SAC and LCQ in real traffic

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPALLI. Downloaded on September 16,2026 at 19:42:48 UTC from IEEE Xplore. Restrictions apply.

V. CONCLUSIONS

In this paper, we develop a model-free DRL framework for the dynamic bandwidth allocation problem in a weighted fair queueing system. The proposed WFQ-DRL framework allows the system to obtain a control policy that reduces both average delay and packet loss rate with limited bandwidth resources. The performance of our trained controllers can converge to the optimal performance when the $c\mu$-rule is valid in certain scenarios and outperform another rule-based policy, LCQ, in real traffic. The WFQ-DRL framework can be applied to achieve efficient utilization of bandwidth resources in telecommunication networks with limited bandwidth, heavy-complex traffic and finite buffer, which is consistent with the characteristics of real routers. An interesting future direction is to extend the DRL framework to multi-agent queueing systems where each agent can only observe its own correspondingly partial state information.

REFERENCES

[1] Y. Ding, Y. Huang, G. Zeng, and L. Xiao, "Using partially overlapping channels to improve throughput in wireless mesh networks," *IEEE Transactions on Mobile Computing*, vol. 11, no. 11, pp. 1720–1733, 2011.

[2] M. Bachl, T. Zseby, and J. Fabini, "Rax: Deep reinforcement learning for congestion control," in *2019 IEEE International Conference on Communications (ICC)*. IEEE, 2019, pp. 1–6.

[3] H. Lee and D. C. Cox, "A fully cooperative and distributed medium access control protocol for large TDMA/TDD-based wireless mesh networks," in *IEEE 69th Vehicular Technology Conference*. IEEE, 2009, pp. 1–7.

[4] K. O. Okokpujie, E. C. Chukwu, E. Noma-Osaghae, and I. P. Okokpujie, "Novel active queue management scheme for routers in wireless networks," *International Journal on Communications Antenna and Propagation (I. Re. CAP)*, vol. 8, no. 1, pp. 53–61, 2018.

[5] L. Zhang, "A new architecture for packet switching network protocols," *Massachusetts Inst of Tech Cambridge Lab for Computer Science*, Tech. Rep., 1989.

[6] A. Demers, S. Keshav, and S. Shenker, "Analysis and simulation of a fair queueing algorithm," *ACM SIGCOMM Computer Communication Review*, vol. 19, no. 4, pp. 1–12, 1989.

[7] M. E. G. Mustafa, S. A. Talab *et al.*, "The effect of queuing mechanisms first in first out (FIFO), priority queuing (PQ) and weighted fair queuing (WFQ) on network's routers and applications," *Wireless Sensor Network*, vol. 8, no. 05, p. 77, 2016.

[8] M. Shreedhar and G. Varghese, "Efficient fair queuing using deficit round-robin," *IEEE/ACM Transactions on Networking*, vol. 4, no. 3, pp. 375–385, 1996.

[9] A. K. Parekh and R. G. Gallager, "A generalized processor sharing approach to flow control in integrated services networks: the single-node case," *IEEE/ACM Transactions on Networking*, vol. 1, no. 3, pp. 344–357, 1993.

[10] G. Horváth and M. Telek, "An approximate analysis of two class WFQ systems," in *Workshop on Preformability Modeling of Computer and Communication Systems-PMCCS*. Citeseer, 2003, pp. 43–46.

[11] F. Guillemin and D. Pinchon, "Analysis of the weighted fair queuing system with two classes of customers with exponential service times," *Journal of Applied Probability*, vol. 41, pp. 832–858, 2004.

[12] A. Al-Sawaai, I. Awan, and R. Fretwell, "Analysis of the weighted fair queuing system with two classes of customers with finite buffer," in *2009 International Conference on Advanced Information Networking and Applications Workshops*. IEEE, 2009, pp. 218–223.

[13] P. Wang, Z. Jiang, M. Qi, L. Dai, and H. Xu, "Uncertainty-aware weighted fair queueing for routers based on deep reinforcement learning," in *2021 IEEE 4th International Conference on Electronics and Communication Engineering*. IEEE, 2021, pp. 1–7.

[14] B. Liu, Q. Xie, and E. Modiano, "RL-QN: a reinforcement learning framework for optimal control of queueing systems," *arXiv preprint arXiv:2011.07401*, 2020.

[15] J. G. Dai and M. Gluzman, "Queueing network controls via deep reinforcement learning," *Stochastic Systems*, 2021.

[16] M. Raeis, A. Tizghadam, and A. Leon-Garcia, "Reinforcement learning-based admission control in delay-sensitive service systems," in *2020 IEEE Global Communications Conference*. IEEE, 2020, pp. 1–6.

[17] M. Raeis, A. Tizghadam, and A. Leon-Garcia, "Queue-learning: A reinforcement learning approach for providing quality of service," *arXiv preprint arXiv:2101.04627*, 2021.

[18] J. D. Little, "A proof for the queuing formula: L= λ w," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[19] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[20] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1861–1870.

[21] L. Xia, Q.-M. He, and A. S. Alfa, "Optimal control of state-dependent service rates in a MAP/M/1 queue," *IEEE Transactions on Automatic Control*, vol. 62, no. 10, pp. 4965–4979, 2017.

[22] J. Baras, D.-J. Ma, and A. Makowski, "K competing queues with geometric service requirements and linear costs: The μc-rule is always optimal," *Systems & Control Letters*, vol. 6, no. 3, pp. 173–180, 1985.

[23] L. Tassiulas and A. Ephremides, "Dynamic server allocation to parallel queues with randomly varying connectivity," *IEEE Transactions on Information Theory*, vol. 39, no. 2, pp. 466–478, 1993.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY TIRUCHIRAPALLI. Downloaded on September 16,2026 at 19:42:48 UTC from IEEE Xplore. Restrictions apply.